Week 3 Discussion

CSE 11 Winter 2026

Arrays

Question: When would it be beneficial to use one form of initializing an array vs. the other?

Initializing arrays

```
double[] myList = new double[3]; // declare
// initialize
myList[0] = 2.0;
myList[1] = 5.6;
myList[2] = 3.8;

double[] anotherList = {3.2, 1.7, 10.3}; // declare and initialize
```

Looping through elements of a 1d array

```
String[] names = {"Tom", "Blake", "Marta", "Zach"};
for (int i = 0; i < names.length; i++) {
    System.out.println("Hi " + names[i] + "!");
}
```

Populating New Arrays

```
record Menu(int[] pricesCents) {
  int[] discounted() {
    int[] result = new int[pricesCents.length];
    int index = 0;
    for(int p: pricesCents) {
      result[index] = p * 90 / 100;
      index += 1;
    }
    return result;
  }
}
```

1. Declare a new result array
2. Declare an index variable
3. Loop over existing array
4. Add result to new array
5. Update index

Exercise: Fix the code

This program should print `rhymes` in reverse order

```
1  public static void main(String args) {
2      String[] rhymes = { "cat", "bat", "hat", "sat" "mat" };
3
4      for (i = rhymes.length(), i > 0, i++) {
5          System.out.print(rhymes[j]);
6      }
7
8      return rhymes[i];
```

Hint: there are 10+ things I would change! Think about errors and good code

Command Line Arguments

```
class TestMain {
  public static void main(String[] args) {
  ...
  }
}
```

```
java TestMain arg0 arg1 arg2 ... argn
```

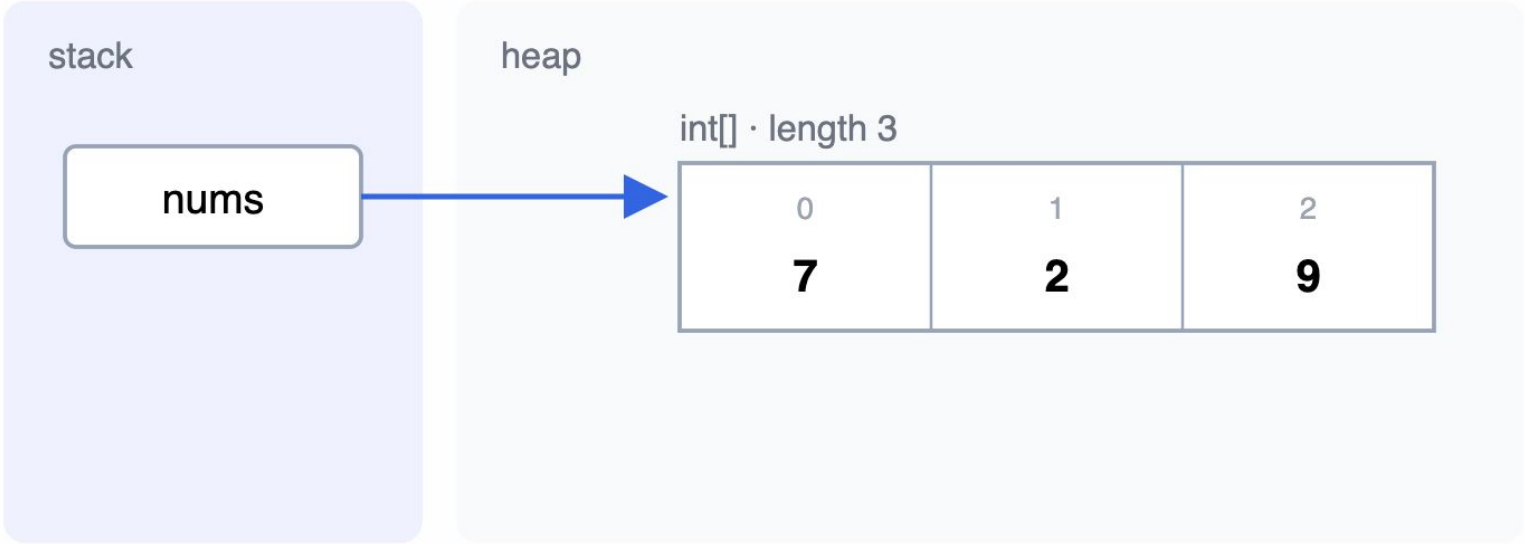
```
public static void main(String args[]) {
    System.out.println(args[0]);
    int iterations = Integer.parseInt(args[1]);
    for (int i = 0; i < iterations; i++) {
        System.out.println("Hello, world!");
}}
```

Concepts

Stack vs. heap	Typically, method calls and local variables are represented on the stack, and objects live on the heap
Primitives vs. references	Primitive variables hold actual values Reference variables hold references to objects
`new` keyword	Creates a new object on the heap
Assignment =	Copies a value Pict a = new Rect(10, 20, “red”); Pict b = a; // copies the reference
Aliasing	Multiple variables can refer to the same object E.g. Picts a and b from above

A one-dimensional array

```
int[] nums = { 7, 2, 9 };
```



> The variable **nums** holds a *reference* (the arrow) to the array object — not the array itself.

Returning an object

```
Gear lighter(Gear a, Gear b) {
  if (a.weightKg() < b.weightKg()) { return a; }
  return b;
}

Gear tent   = new Gear("tent", 2.3);
Gear jacket = new Gear("jacket", 0.9);
Gear winner = lighter(tent, jacket);
```

```
record Gear(String name, double weightKg) {
  boolean isLight() { return weightKg < 1.0; }
}
```

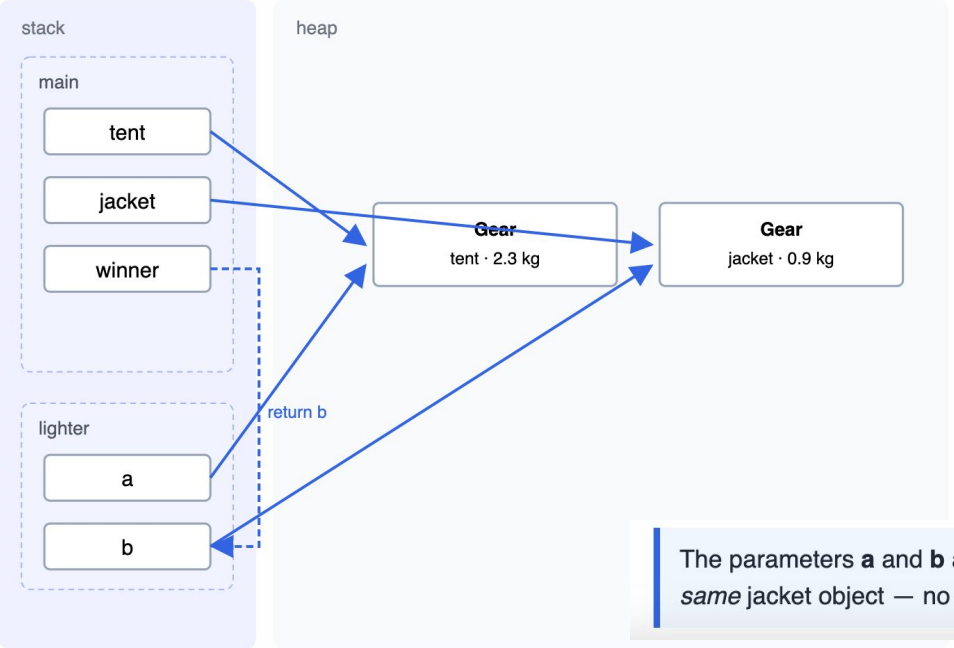
Question (5 min) : Draw the memory diagram after this code has finished running. How many objects are created?

Returning an object

```
Gear lighter(Gear a, Gear b) {
  if (a.weightKg   < b.weightKg  ) { return a; }
  return b;
}

Gear tent   = new Gear("tent", 2.3);
Gear jacket = new Gear("jacket", 0.9);
Gear winner = lighter(tent, jacket);
```

```
record Gear(String name, double weightKg) {
  boolean isLight() { return weightKg < 1.0; }
}
```



* Simplified heap without String allocation. “Tent” and “jacket” would exist on the heap outside of their Gear objects

> The parameters **a** and **b** are copies of the references, so **return b** hands back a reference to the *same* jacket object — no new Gear is created, and **winner** and **jacket** now share it.

In class exercise (10 min)

1. Pull up your solution to the **checkerboard problem from PA01** on PrairieLearn
2. To the best of your ability, draw the memory diagram after all your code has executed.
3. Disregard the call to `ImageIO_.save()`
4. **How many new objects did you create on the heap?**

In class exercise (10 min)

1. Pull up your solution to the **checkerboard problem from PA01** on PrairieLearn
2. To the best of your ability, draw the memory diagram after all your code has executed.
3. Disregard the call to `ImageIO_.save()`
4. **How many new objects did you create on the heap?**

```
void main(){
    Pict dot1 = new Circle(20, "pink");
    Pict dot2 = new Circle(20, "transparent");

    Pict dots1 = dot1.beside(dot2);
    Pict dots2 = dot2.beside(dot1);

    Pict dots = dots1.beside(dots1).above(
        dots2.beside(dots2);
    );

    ImageIO_.save(dots, "polkadots.png");
}
```

In class exercise (10 min)

1. Pull up your solution to the **checkerboard problem from PA01** on PrairieLearn
2. To the best of your ability, draw the memory diagram after all your code has executed.
3. Disregard the call to `ImageIO_.save()`
4. **How many new new objects did you create on the heap?**

Example solution from drawing polka dots from last week's discussion section

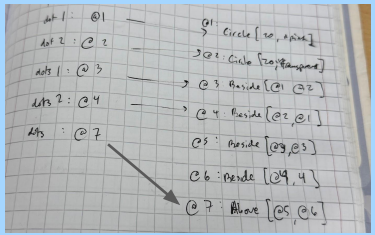
```
void main(){
    Pict dot1 = new Circle(20, "pink");
    Pict dot2 = new Circle(20, "transparent");

    Pict dots1 = dot1.beside(dot2);
    Pict dots2 = dot2.beside(dot1);

    Pict dots = dots1.beside(dots1).above(
        dots2.beside(dots2);
    );

    ImageIO_.save(dots, "polkadots.png");
}
```

What was drawn on the board (simplified for ease of writing):



Thank You!
Any Questions?